

Name : Manaswi Kabadi

Class : D20C

Roll no. : 15

Blockchain Experiment - 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. Hash Generation using SHA-256:

- Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. Target Hash Generation with Nonce:

- Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. Proof-of-Work Puzzle Solving:

- Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. Merkle Tree Construction:

- Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output :

```
1. import hashlib
def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string
# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Enter a string: Manaswi Kabadi

Hash: c2831a8be6c8e81161537c00680458d5dc0df34d56519ba2f86a2e5403bb9eb3

```
2. import hashlib
# Get user input
```

```

input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
# Concatenate the string and nonce
hash_string = input_string + nonce
# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()
# Print the hash code
print("Hash Code:", hash_code)

```

```

Enter a string: Manaswi
Enter the nonce: 4
Hash Code: 614eb2b4e953e55640b908d82cce58bcfdd45c77a0c686de25712bfaf8f6a5f3

```

```

3. import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()
        # Check if the hash has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce
        nonce += 1
# Get user input
input_string = "Manaswi"
num_zeros = 1
# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)
# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

```

Hash: 08bd152c71b0266bde86e9a50d08f49e5c9d6007185e647c08d52eaab3544a1b
Input String: Manaswi
Leading Zeros: 1
Expected Nonce: 15

```

```

4. import hashlib
def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None
    level = 0
    while len(transactions) > 1:
        print("\n===== Level",
level, "=====")
        # If odd number of
transactions, duplicate the last one
        if len(transactions) % 2 !=
0:
transactions.append(transactions[-1]
)
        new_transactions = []
        for i in range(0,
len(transactions), 2):
            # Hash each transaction
(or previous level hash)
            hash1 =
hashlib.sha256(transactions[i].encod
e('utf-8')).hexdigest()
            hash2 =
hashlib.sha256(transactions[i +
1].encode('utf-8')).hexdigest()
            print("Hash 1 :", hash1)
            print("Hash 2 :", hash2)
            # Combine the hashes
            combined = hash1 + hash2
            hash_combined =
hashlib.sha256(combined.encode('utf-
8')).hexdigest()
            print("Combined Hash :",
hash_combined)

```

```

        print()
new_transactions.append(hash_combine
d)
        transactions =
new_transactions
        level += 1
        return transactions[0]
# ----- Main Program
-----
transactions = [
    "Manaswi -> Saniya : $100",
    "Saniya -> Atharva : $300",
    "Atharva -> Mansi : $250",
    "Mansi -> Bhoomi : $400",
    "Bhoomi -> Atharva : $50"]
print("Transactions:")
for t in transactions:
    print(t)

merkle_root =
build_merkle_tree(transactions)
print("\n=====
====")
print("Final Merkle Root:",
merkle_root)
print("=====
====")
merkle_root =
build_merkle_tree(transactions)
print("\n=====
====")
print("Final Merkle Root:",
merkle_root)
print("=====
====")

```

```

Transactions:
Manaswi -> Saniya : $100
Saniya -> Atharva : $300
Atharva -> Mansi : $250
Mansi -> Bhoomi : $400
Bhoomi -> Atharva : $50

===== Level 0 =====
Hash 1 : ae527f16435418702f2e6e94f89075fe6094a8ecbadd28b616f6923195ed6a63
Hash 2 : b69e3d642223b4c34bd60d72039fd3b5306458beb3d946bb737b8d3036b9a699
Combined Hash : 70332c012c658cd39b16f466510438105c50c6c7997d291a3cef7831055f578f

Hash 1 : ab47da7cfff05567f5a04105f7e8365bba8044324b65783230cc5168daa144857
Hash 2 : d425b1ff02d5237e62cc854441122a9fc6ec9ab748f99c3ddab5ec19acbc153d
Combined Hash : 74c4ac89ef3686f94e0873a8e5f7bfe3fe4dc65b61cf976844f36dda4a017e74

Hash 1 : a0fe243835dd53a19fc8d4c1ee615dfb7d3f6723b6d57c8d7e20ad2d2f6f3878
Hash 2 : a0fe243835dd53a19fc8d4c1ee615dfb7d3f6723b6d57c8d7e20ad2d2f6f3878
Combined Hash : ef15356f18daec77884cbdac7884b14380f0ceed470ab2f61d051fd47c5299

```

```

===== Level 1 =====
Hash 1 : c09c3443303fdbf4ba8b5566d32f023bec7a395e70a7ec14fc3e6ef8b0ad4785
Hash 2 : e6367ccc20640984042d5ac8b44ef78f776a95bcf262a4df52376f9850584b33
Combined Hash : 10386b87e09ee881549c984bbdf372312e8c18cdfd0d53036bc736c2dcd7222f

Hash 1 : d9c916dd1da0d975107aa060effebf5da0717160b4fab86681c587b2115927aa
Hash 2 : d9c916dd1da0d975107aa060effebf5da0717160b4fab86681c587b2115927aa
Combined Hash : 1ef81c071c1bf390c96218d737b3f58e4f163ddc377b727f2333ad23f6d8e0a5

===== Level 2 =====
Hash 1 : 9decc7876e0f0a4b630913e44349e7df25e807fdf9c638680734eb3ee07be162
Hash 2 : bd689db44409b37a111831678f3ff92f82db9b2793f8bb1a4eb986b43e915966
Combined Hash : e654baaf243926ac68f23afb5d797a971eafe0f98efc870def02a11d001888d0

=====
Final Merkle Root: e654baaf243926ac68f23afb5d797a971eafe0f98efc870def02a11d001888d0
=====

```

Conclusion :

This experiment demonstrated the fundamental concepts of cryptographic hashing and its applications in blockchain technology through four programs. The first program showed how to generate a SHA-256 hash for a given string, while the second program illustrated the use of a nonce in hash generation. The third program implemented a simple Proof of Work mechanism by finding a nonce that produces a hash with the required number of leading zeros. Finally, the fourth program constructed a Merkle Tree and generated the Merkle Root by hashing and combining transaction data. Together, these programs highlight how cryptographic hashing ensures data integrity, authenticity, and security, and how techniques such as Proof of Work and Merkle Trees play a crucial role in maintaining the reliability and tamper-resistant nature of blockchain systems.